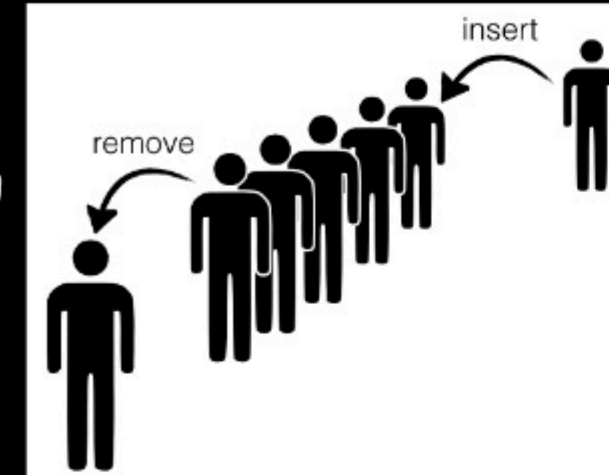


# Queue

## Agenda :

- ✓ 1. What is a Queue? ←
- ✓ 2. Queue ADT ←
- ✓ 3. Common operations on a Queue ✓
- ✓ 4. Types of Queue (linear, circular, double ended, priority queue) ✓
- ✓ 5. Applications of Queue ✓
- ✓ 6. Implementations of Queue in Python ✓
- ✓ 7. Linear Queue v/s Circular Queue ✓
- ✓ 8. Some MCQs on Queue
- ✓ 9. Leetcode problems on Queue

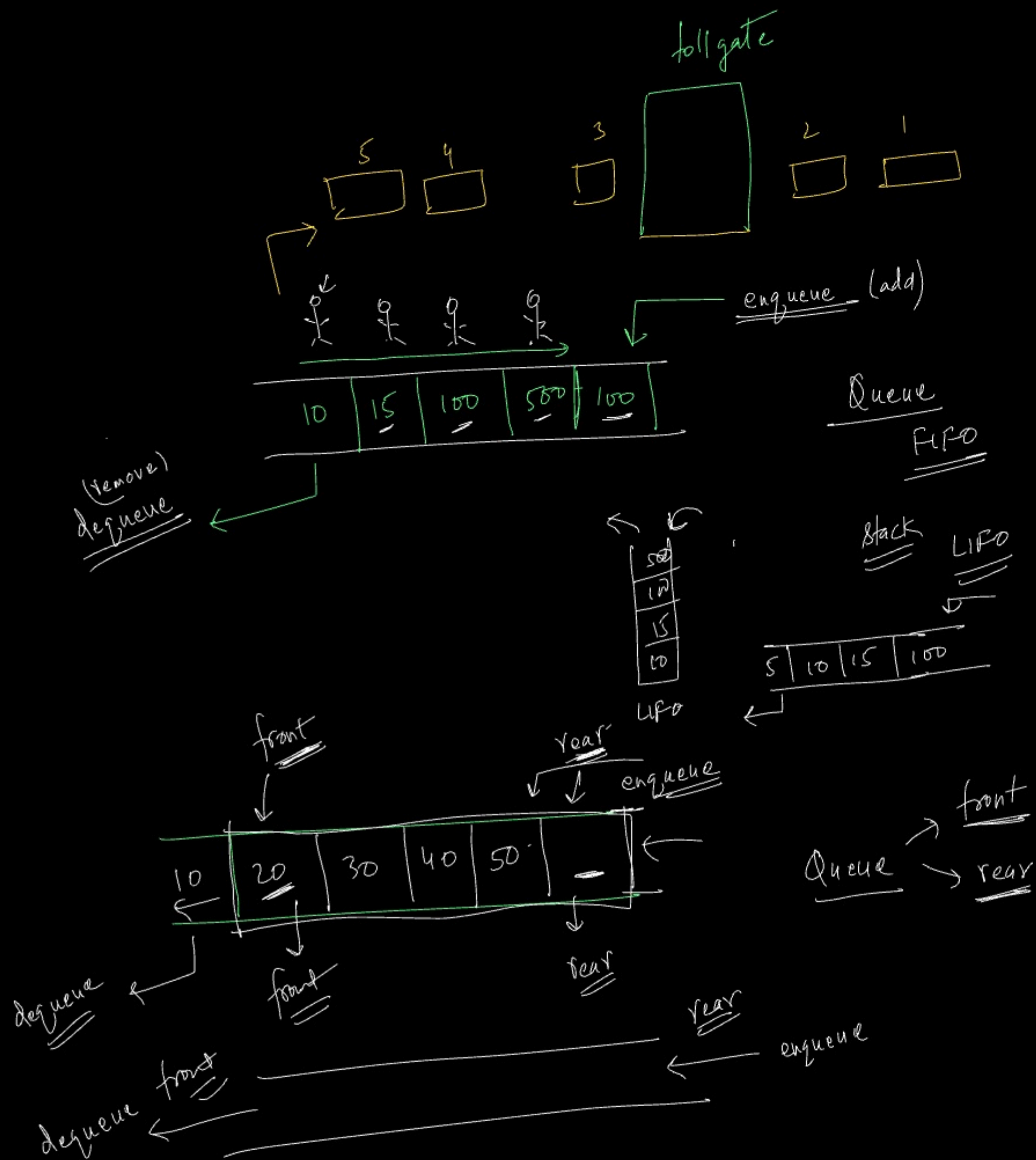


✓ deque ✓

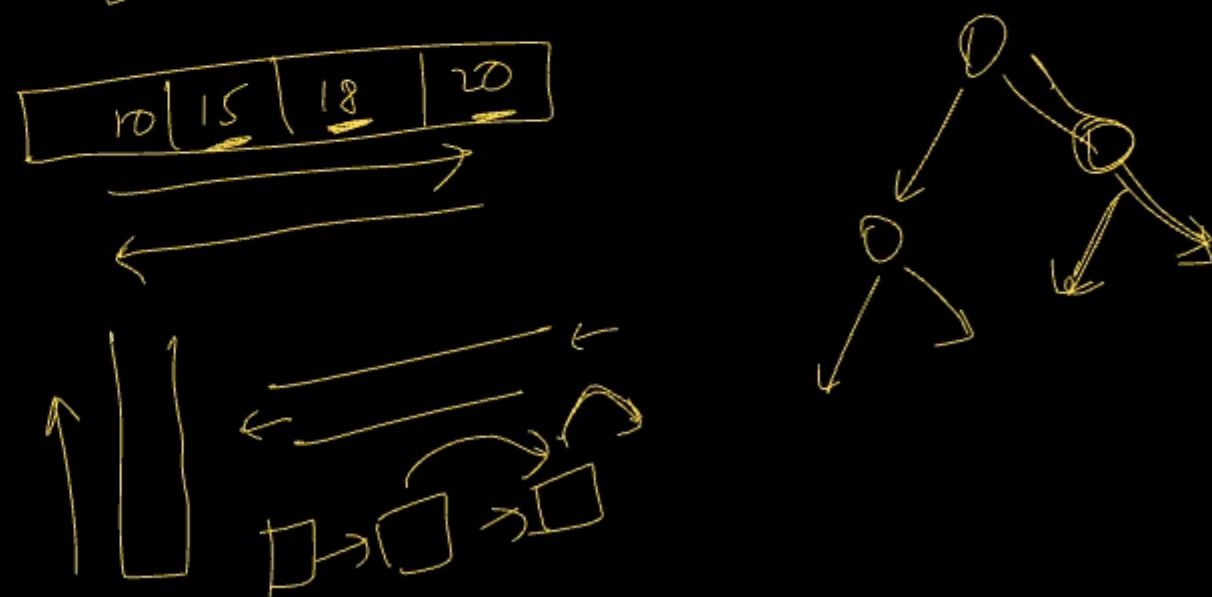
## What is a Queue?

- A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle.
- Elements are always inserted at the rear and removed from the front.
- It supports basic operations like enqueue, dequeue, peek, and checks for empty or full states.
- Queues can be implemented using arrays, linked lists, or built-in structures such as deque in Python.
- Common types of queues include linear queues, circular queues, double-ended queues (deque), and priority queues.
- Queues are used in operating systems for process scheduling, in networking for packet buffering, in printers and spooling systems, in graph algorithms like BFS, and in solving producer-consumer problems.
- Real-world examples of queues include waiting lines at ticket counters, supermarket checkouts, call center waiting lists, and traffic at signals.

Queue → collection of items  
 → linear data structure  
 → FIFO principle



| linear data structures                 | Non linear data structures                   |
|----------------------------------------|----------------------------------------------|
| array<br>stack<br>Queue<br>linked list | - Graphs<br>- Trees<br>- Hash Maps<br>- Heap |



### Common Operations on Queue (Queue ADT)

- Queue (ADT)
- enqueue →  $O(1)$  ✓
  - dequeue →  $O(1)$  ✓
  - front & rear →  $O(1)$  ✓
  - is-empty, is-full →  $O(1)$  ✓
  - size →  $O(1)$  ✓









$$5 \rightarrow (5+1) \% 6$$

## Circular Queue using Array

```
1  ## Circular Queue using Array
2
3  class MyCircularQueue:
4      def __init__(self, k = 5):
5          self.capacity = k
6          self._data = [None] * self.capacity
7          self._size = 0
8          self._front = 0
9          self._rear = 0
10
11     def isFull(self):
12         return self._size == self.capacity
13
14     def isEmpty(self):
15         return self._size == 0
16
17     def size(self):
18         return self._size
19
20     def enQueue(self, val):
21         '''
22         This method will add an element to the queue
23         TC: O(1)
24         '''
25         if self.isFull(): # if the queue is full
26             return False
27         self._data[self._rear] = val
28         self._rear = (self._rear + 1) % self.capacity
29         self._size = self._size + 1
30         return True
31
32     def deQueue(self):
33         if self.isEmpty():
34             return False
35
36         val = self._data[self._front]
37         self._data[self._front] = None # set the popped element to None
38         self._front = (self._front + 1) % self.capacity # wrap using modulo division
39         self._size = self._size - 1
40         return True
41
42     def Front(self):
43         if self.isEmpty():
44             return -1
45         return self._data[self._front]
46
47     def Rear(self):
48         if self.isEmpty():
49             return -1
50         return self._data[self._rear - 1]
51
52     def __str__(self):
53         return str(self._data)
54
55     def __repr__(self):
56         return str(self._data)
57
58
```



## # Circular Queue using linked list

```
1  ## Circular Queue using Linked list ( Leetcode 622 )
2
3  class Node:
4      def __init__(self, data):
5          self.data = data
6          self.next = None
7
8
9  class MyCircularQueue:
10     def __init__(self, k = 5):
11         self.capacity = k
12         self._size = 0
13         self._head = None
14         self._tail = None
15
16     def isFull(self):
17         return self._size == self.capacity
18
19     def isEmpty(self):
20         return self._size == 0
21
22     def size(self):
23         return self._size
24
25     def enqueue(self, val):
26
27         if self.isFull():
28             return False
29
30         node = Node(val)
31
32         if self.isEmpty():
33             self._head = node
34             self._tail = node
35             node.next = node
36
37         else:
38             self._tail.next = node
39             self._tail = node
40             self._tail.next = self._head
41
42         self._size = self._size + 1
43         return True
44
45     def dequeue(self):
46         if self.isEmpty():
47             return False
48
49         if self._size == 1:
50             self._head = None
51             self._tail = None
52
53         else:
54             self._head = self._head.next
55             self._tail.next = self._head
56
57         self._size = self._size - 1
58         return True
59
60
61     def Front(self):
62         if self.isEmpty():
63             return -1
64
65         return self._head.data
66
67     def Rear(self):
68         if self.isEmpty():
69             return -1
70         return self._tail.data
71
72
73     # def to_list(self):
74     #     '''
75     #     TC : O(n)
76     #     '''
77     #     elements = []
78     #     curr = self._head
79
80     #     for _ in range(self._size):
81     #         elements.append(curr.data)
82     #         curr = curr.next
83
84
85     #     return list(elements) # it will return a copy of internal
86     #     list
87
88     # def __str__(self):
89     #     return (" | ").join([str(x) for x in (self.to_list())])
90
91     # def __repr__(self):
92     #     return (" | ").join([str(x) for x in (self.to_list())])
93
94
95
96
97
98
99
```





### Question 1: [GATE (CS) 1996]

Consider the following statements:

- ☒ i. First-in-first-out types of computations are efficiently supported by stacks. LIFO
- ☒ ii. Implementing lists on linked lists is more efficient than implementing lists on an array for almost all the basic list operations.
- ☒ iii. Implementing queues on a circular array is more efficient than implementing queues on a linear array with two indices.
- ☒ iv. Last-in-first-out type of computations are efficiently supported by queues. FIFO

Which of the following is correct?

- ☒ (a) (ii) and (iii) are true
- (b) (i) and (ii) are true ☒
- (c) (iii) and (iv) are true ☒
- (d) (ii) and (iv) are true ☒

Stack → Queue

push  
pop  
reverse

Queue

reverse  
enqueue  
push  
reverse

enqueue  
dequeue

Stack

push  
pop  
reverse

30  
20  
10

10  
20  
30

①

⑤ reverse pop reverse

### Question 2: [GATE (CS) 1996]

Suppose a stack implementation supports an instruction reverse, which reverses the order of elements of the stack, in addition to the push and pop instructions.

Which of the following statements is true with respect to using this stack? —

- ☒ (a) A queue cannot be implemented using this stack. ☒
- ☒ (b) A queue can be implemented where enqueue takes a single instruction and dequeue takes a sequence of two instructions.
- ☒ (c) A queue can be implemented where enqueue takes a sequence of three instructions and dequeue takes a single instruction.
- ☒ (d) A queue can be implemented where both enqueue and dequeue take a single instruction each.

Stack

push

20  
10

30  
10  
20

20  
10  
30

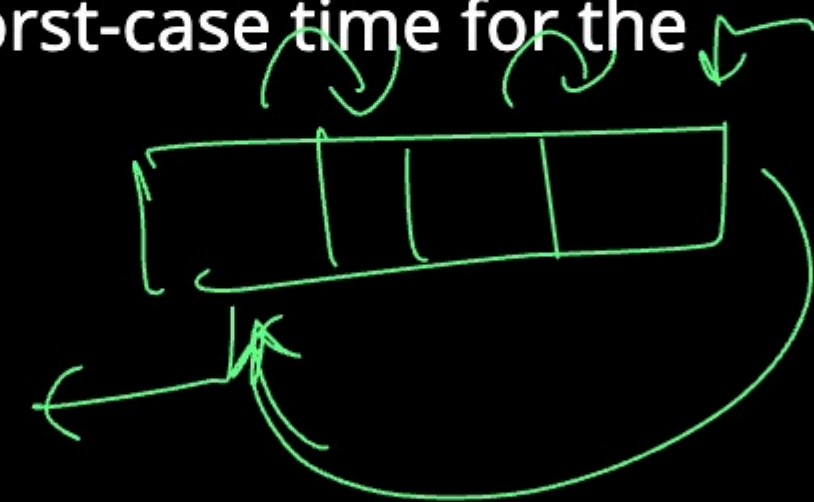
reverse  
push  
reverse

### Question 3: [GATE (CS) 2014]

A queue is implemented using an array such that enqueue and dequeue operations are performed efficiently.

Which of the following statements is correct ( $n$  refers to the number of items in the queue)? ✓

- ✓ (a) Both operations can be performed in  $O(1)$  time. ✓
- ✗ (b) At most one operation can be performed in  $O(1)$  time but the worst-case time for the other operation will be  $\Omega(n)$ .
- ✗ (c) Worst-case time complexity for both operations will be  $\Omega(n)$ .
- ✗ (d) Worst-case time complexity for both operations will be  $\Omega(\log n)$ .



### Question 4: [GATE (CS) 2016]

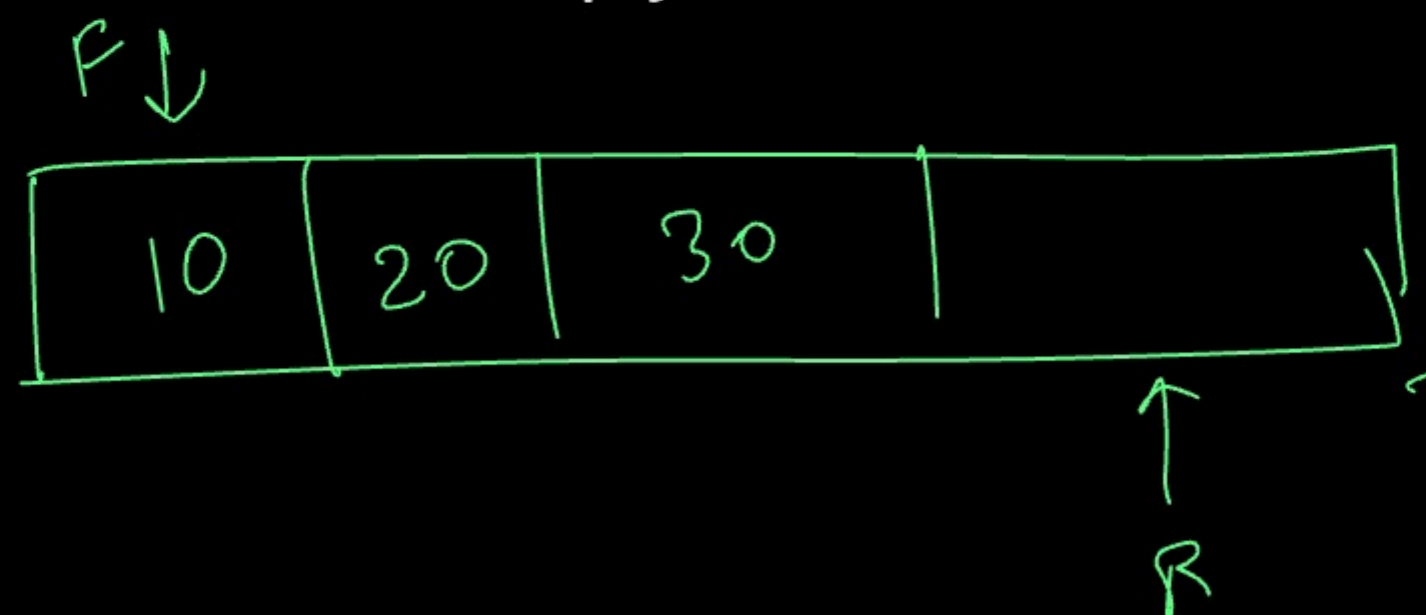
Suppose a circular queue of capacity  $(n - 1)$  elements is implemented with an array of  $n$  elements.

Assume that insertion and deletion operations are carried out using rear and front as array index variables respectively.

Initially, rear = front = 0.

The conditions to detect queue full and queue empty are:

- ✓ (a) Full:  $(rear + 1) \bmod n == front$ , Empty:  $rear == front$  ✓
- (b) Full:  $(rear + 1) \bmod n == front$ , Empty:  $rear (front + 1) \bmod n == front$
- (c) Full:  $(rear) == front$ , Empty:  $(rear + 1) \bmod n == front$
- (d) Full:  $(front + 1) \bmod n == rear$ , Empty:  $rear == front$  ✓



size = 4  
capacity = 3

$$(R+1) \% size = F$$

size = 20  
rear = capacity  
capacity > size

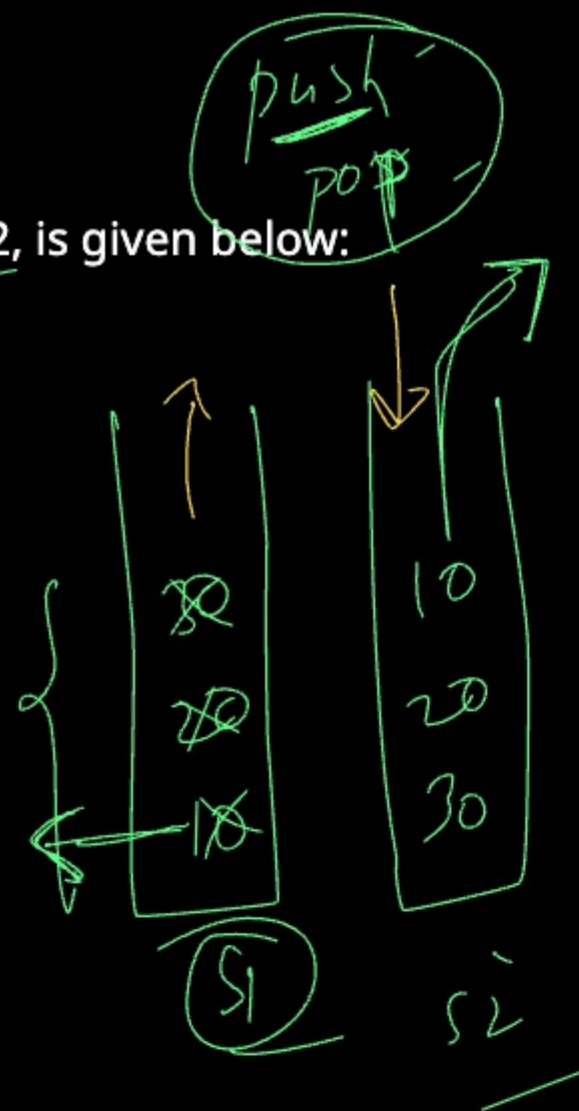
### Question 5: [GATE (CS) 2016]

An implementation of a queue  $Q$ , using two stacks  $S1$  and  $S2$ , is given below:

```
void insert (Q, x) {
    push (S1, x);
}

void delete (Q) {
    if (stack-empty(S2)) then
        if (stack-empty(S1)) then {
            print("Q is empty");
            return;
        }
        else while (!stack-empty(S1)) {
            x = pop(S1);
            push(S2, x);
        }
    x = pop(S2);
}
```

enqueue  
dequeue  
push  $\rightarrow 3+3$   
pop  $\rightarrow 3+3$



insert  $\rightarrow$  push ①  
delete  $\rightarrow$  push  $\rightarrow 3$   
pop  $\rightarrow 3+1$

Best Case

Worst Case

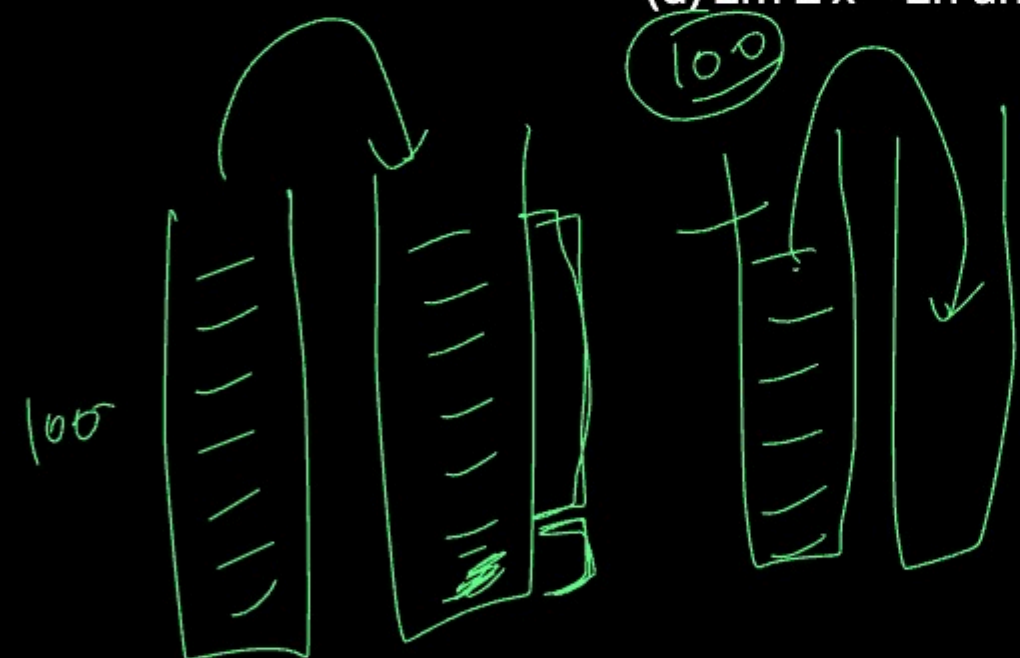
Let  $n$  insert and  $m$  ( $\leq n$ ) delete operations be performed in an arbitrary order on an empty queue.

Let  $x$  and  $y$  be the number of push and pop operations performed respectively in the process.

Which of the following is true for all  $m$  and  $n$ ?

- (a)  $n + m \leq x < 2n$  and  $2m \leq y \leq n + m$
- (b)  $n + m \leq x < 2n$  and  $2m \leq y \leq 2n$
- (c)  $2m \leq x < 2n$  and  $2m \leq y \leq n + m$
- (d)  $2m \leq x < 2n$  and  $2m \leq y \leq 2n$

100 80 delete



Worst Case

Best Case

Case 1  
100 insert (m)  
80 delete (n)

80 insert  
80 delete  
20 insert

x, y  
push pop

100 insert  
80 delete

push  $\rightarrow 100 + 100 \rightarrow 2n$   
pop  $\rightarrow 100 + 80 \rightarrow m + n$

push  $\rightarrow 80 + 80 + 20 \rightarrow m + n$   
pop  $\rightarrow 80 + 80 \rightarrow 2m$

sh

### Question 6: [GATE (CS) 2016]

Consider a standard circular queue  $Q$  implementation (which has the same condition for queue full and queue empty) whose size is 11 and the elements of the queue are  $q[0]$ ,  $q[1]$ ,  $q[2]$ , ...,  $q[10]$ .

The front and rear pointers are initialized to point at  $q[2]$ .

In which position will the ninth element be added?

- (a)  $q[0]$
- (b)  $q[1]$
- (c)  $q[9]$
- (d)  $q[10]$

